

SUB-PROJECT REQUIREMENTS MANAGEMENT

PLAN

SP-RMP

Analytics & Community Insights Module

Verified Neighborhood Community Hub (Bring)

Project	Verified Neighborhood Community Hub (Bring)
Module	Analytics & Community Insights
Created by	Hamad Khan 23I-3095
Group	Koders
Integration Group	Hassan Qureshi, Ahmed Ali, M. Irfan, M. Hamza
Version / Date	1.0 30 March 2026

National University of Computer & Emerging Sciences (NUCES)

1. Introduction

1.1 Purpose

This Sub-Project Requirements Management Plan (SP-RMP) defines the process for identifying, documenting, tracking, and managing requirements for the Analytics & Community Insights module of the Verified Neighborhood Community Hub (Bring). It serves as the governance document for all requirements activities within this module and provides the basis for the SP-SRS.

1.2 Scope

This plan applies exclusively to the Analytics & Community Insights module. It covers all functional and non-functional requirements related to event data ingestion, aggregation, reporting, and the admin-facing dashboard. Requirements for other modules (authentication, posts, marketplace, etc.) are out of scope for this plan.

1.3 Definitions and Acronyms

Term / Acronym	Definition
SP-RMP	Sub-Project Requirements Management Plan
SP-SRS	Sub-Project Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
MERN	MongoDB, Express.js, React.js, Node.js -- mandated technology stack per Project Charter
User ID	Shared unique user identifier used across all Bring modules
Neighborhood ID	Shared unique neighborhood identifier used across all Bring modules
Event	A user action or system occurrence published by any Bring module to the Analytics ingestion API
KPI	Key Performance Indicator -- a measurable value tracked in the analytics dashboard
CHS	Community Health Score -- composite score (0-100) derived from engagement, safety, and moderation metrics
ClickUp	Primary project management and communication tool designated in the Project Charter
RTM	Requirements Traceability Matrix -- maps each requirement to its source, WBS item, and verification method

1.4 References

- Project Charter – Verified Neighborhood Community Hub (Bring), Section A, 12 March 2026
- Assignment 2 Requirements – NUCES, Section A

- SPMP – Analytics & Community Insights Module, v2.0, 27 March 2026
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications
- Pressman, R. S. – Software Engineering: A Practitioner's Approach (textbook)

2. Requirements Management Overview

2.1 Requirements Management Approach

Requirements for the Analytics & Community Insights module are managed through a structured, team-based process aligned with the 6-week academic project timeline. The approach is organized around four distinct team roles — Team Lead, Backend Developer, Frontend Developer, and QA Engineer — each with defined responsibilities in the requirements lifecycle.

- Requirements are elicited from the Project Charter objectives, the module scope defined in the SPMP, and integration contracts agreed with the Integration Group.
- All requirements are documented in the SP-SRS with unique identifiers to enable traceability across the team.
- Requirements are reviewed collectively by the team and baselined after the kickoff meeting. Changes are managed through a formal change control process.
- Traceability is maintained from requirements through WBS work packages, team member assignments, and test cases.
- ClickUp is used as the primary tool for tracking requirement status, task assignments, and cross-team communication.

2.2 Team Roles in Requirements Management

Team Member	Role	Requirements Responsibilities
Shah Faisal (23I-0058)	Team Lead	Owns overall requirements baseline; coordinates elicitation with Integration Group; approves all requirements changes; ensures traceability between requirements, WBS, and schedule
Shace Tariq (23I-3069)	Backend Developer	Provides technical input on backend feasibility of FR-EI and FR-AG requirements; raises change requests when implementation reveals gaps; owns backend-related NFRs (performance, reliability, security)
Ali Sajjad (23I-0110)	Frontend Developer	Provides input on feasibility of FR-DB dashboard requirements; reviews UI-related NFRs (usability, compatibility); raises change requests for dashboard requirements as needed
Hamad Khan (23I-3095)	QA Engineer	Reviews all requirements for testability; defines verification methods per requirement; maintains the RTM; leads requirement verification and acceptance testing

2.3 Requirements Organisation

Requirements are organised into two categories:

- **Functional Requirements (FR):** describe specific behaviours, data processing, and outputs of the Analytics module.
- **Non-Functional Requirements (NFR):** describe quality attributes including performance, security, usability, reliability, and technology constraints.

Within functional requirements, requirements are grouped by feature area: Event Ingestion (EI), Data Aggregation (AG), Analytics API (AP), Dashboard UI (DB), and Reporting (RP). Each area maps to a corresponding WBS work package and team member responsibility.

2.4 Requirements Identification Scheme

Each requirement is assigned a unique identifier using the following format:

Format	Meaning	Example
FR-EI-XXX	Functional – Event Ingestion	FR-EI-001: System shall accept POST /api/events
FR-AG-XXX	Functional – Data Aggregation	FR-AG-001: System shall compute daily active users
FR-AP-XXX	Functional – Analytics API	FR-AP-001: System shall expose GET /api/analytics/summary
FR-DB-XXX	Functional – Dashboard UI	FR-DB-001: Dashboard shall display engagement charts
FR-RP-XXX	Functional – Reporting	FR-RP-001: System shall support CSV export
NFR-XXX	Non-Functional Requirement	NFR-001: API response time < 2 seconds

This SP-RMP is aligned with the SPMP and ensures that all requirements are directly traceable to WBS work packages, schedule baseline, and team responsibilities.

3. Requirements Elicitation

3.1 Sources of Requirements

Source	Description
Project Charter	Defines the objective of delivering analytics and insights for admins and moderators; mandates MERN stack and zero-budget constraint; names shared identifiers (User ID, Neighborhood ID)
SPMP (Module Scope)	Defines in-scope and out-of-scope boundaries; provides WBS work packages that map directly to requirements and team member task assignments
Kickoff Meeting	Agreed module boundaries, shared data identifiers, integration points, and API contract freeze date (31 March 2026); documented in kickoff meeting minutes
Integration Group API Contracts	Define the interface this module must expose and the event schema it must consume from other modules; frozen after 31 March 2026
Business Case (Assignment 1)	Identifies platform pain points (spam, misinformation, poor safety coordination) that analytics must address through measurable community insights
Platform Admin / Moderator (Implied Users)	Administrators need to monitor neighbourhood health, user engagement, incident trends, and module activity; moderators need neighbourhood-level safety reports

3.2 Elicitation Techniques Used

Technique	Description	Applied By
Document Analysis	Review of Project Charter, SPMP, and business case to derive module objectives and constraints	Shah Faisal (Team Lead)
Interface Analysis	Analysis of Integration Group API contracts and shared identifier schema to derive ingestion requirements	Shace Tariq (Backend Dev)
Goal Decomposition	Module objectives decomposed into specific, measurable functional behaviours per WBS work package	Whole Team
ClickUp Collaboration	Requirements reviewed and agreed with Integration Group on ClickUp before baseline; feedback incorporated	Shah Faisal (Team Lead)
Testability Review	Each drafted requirement reviewed by QA Engineer to confirm it can be verified; ambiguous requirements revised	Hamad Khan (QA Engineer)

4. Requirements Documentation

4.1 Requirements Attributes

Every requirement in the SP-SRS is documented with the following attributes to ensure completeness, clarity, and traceability across the team:

Attribute	Description
ID	Unique identifier (e.g., FR-EI-001) used in ClickUp tasks, RTM, and team discussions
Title	Short descriptive name for quick reference
Description	Clear, unambiguous statement of what the system shall do, written using 'shall' language
Priority	High / Medium / Low -- based on impact on integration and user value; agreed by team and team lead
Source	Origin of the requirement (Charter, SPMP, API Contract, Kickoff Meeting, etc.)
WBS Reference	Corresponding WBS work package ID from the SPMP; used to assign to team members
Owner	Team member responsible for implementation (Backend Dev, Frontend Dev, or QA Engineer)
Status	Draft / Baselined / Changed / Implemented / Verified -- tracked on ClickUp
Verification Method	How compliance will be verified: Unit Test / Integration Test / Demo / Performance Test / Inspection / Analysis

4.2 Requirements Baseline

Requirements are baselined after the kickoff meeting (18 March 2026). The baselined SP-SRS becomes the reference document for all development, integration testing, and grading. The Team Lead (Shah Faisal) is responsible for maintaining the baseline and approving changes. Status of each requirement is tracked on ClickUp.

4.3 Requirements by Team Owner

Each requirement category is assigned to a primary team owner responsible for implementation. The QA Engineer (Hamad Khan) is responsible for verification across all categories.

Requirement Area	Primary Owner	Verification Owner
FR-EI: Event Ingestion	Shace Tariq – Backend Developer	Hamad Khan – QA Engineer
FR-AG: Data Aggregation	Shace Tariq – Backend Developer	Hamad Khan – QA Engineer
FR-AP: Analytics API	Shace Tariq – Backend Developer	Hamad Khan – QA Engineer
FR-DB: Dashboard UI	Ali Sajjad – Frontend Developer	Hamad Khan – QA Engineer
FR-RP: Reporting / Export	Ali Sajjad – Frontend Developer	Hamad Khan – QA Engineer
NFR: Non-Functional	Whole Team (per area)	Hamad Khan – QA Engineer

5. Requirements Traceability

5.1 Purpose of the RTM

The Requirements Traceability Matrix (RTM) ensures every requirement can be traced from its source through to design, team assignment, implementation (WBS), and verification. The RTM is maintained by the QA Engineer (Hamad Khan) and updated whenever a requirement is added, changed, or verified.

5.2 Requirements Traceability Matrix

Req. ID	Requirement Summary	Source	WBS Ref.	Owner	Verification
FR-EI-001	Accept POST /api/events from any module	API Contract	1.2.2	Shah Faisal	Integration Test
FR-EI-002	Validate User ID & Neighborhood ID on ingestion	Charter	1.2.2	Shah Faisal	Unit Test
FR-EI-003	Store validated events in MongoDB events collection	SPMP	1.2.1/1.2.2	Shah Faisal	Unit Test
FR-EI-004	Support optional metadata field in event payload	API Contract	1.2.2	Shah Faisal	Unit Test
FR-AG-001	Compute daily active users (DAU) per neighborhood	Charter	1.2.3	Shah Faisal	Unit Test
FR-AG-002	Compute post and incident counts per day	Charter / SPMP	1.2.3	Shah Faisal	Unit Test
FR-AG-003	Compute Community Health Score (CHS) weekly	SPMP	1.2.5	Shah Faisal	Analysis
FR-AP-001	Expose GET /api/analytics/summary	API Contract	1.2.4	Shah Faisal	Integration Test
FR-AP-002	Expose GET /api/analytics/neighborhood/:id	API Contract	1.2.4	Shah Faisal	Integration Test
FR-AP-003	Expose GET /api/analytics/incidents	API Contract	1.2.4	Shah Faisal	Integration Test
FR-AP-004	Expose GET /api/analytics/export (CSV download)	SPMP	1.3.5	Shah Faisal	Test
FR-AP-005	Detect anomalies and log alerts to MongoDB	SPMP	1.2.6	Shah Faisal	Unit Test
FR-DB-001	React dashboard with sidebar layout and navigation	Charter	1.3.1	Ali Sajjad	Demo

Req. ID	Requirement Summary	Source	WBS Ref.	Owner	Verification
FR-DB-002	Engagement charts (DAU, posts, registrations)	Charter	1.3.2	Ali Sajjad	Demo
FR-DB-003	Neighborhood selector dropdown updating all charts	SPMP	1.3.1	Ali Sajjad	Demo
FR-DB-004	Safety and incident trend charts	Charter	1.3.3	Ali Sajjad	Demo
FR-DB-005	Community Health Score colored display	SPMP	1.3.2	Ali Sajjad	Demo
FR-DB-006	Module KPI summary cards per Bring module	SPMP	1.3.4	Ali Sajjad	Demo
FR-DB-007	CSV export UI with date range picker	SPMP	1.3.5	Ali Sajjad	Test
NFR-001	API response time < 2 seconds	Charter / NFR	1.4.1	Shah Faisal	Performance Test
NFR-002	Dashboard load < 4 seconds	NFR	1.3.*	Ali Sajjad	Performance Test
NFR-003	No hardcoded secrets; environment variables only	Charter	All	Shah Faisal	Inspection
NFR-004	MERN stack compliance throughout	Charter	All	Whole Team	Inspection
NFR-005	Zero paid tools, APIs, or services	Charter	All	Whole Team	Inspection
NFR-006	Dashboard renders in Chrome and Firefox at 1280px+	NFR	1.3.*	Hamad Khan	Manual Test

6. Requirements Change Management

6.1 Change Control Process

Requirements changes after the baseline (18 March 2026) must follow this process. The Team Lead (Shah Faisal) is the final approver for all requirements changes within the module. Changes affecting API contracts additionally require Integration Group approval.

Step	Activity	Description	Responsibility
1	Identify Change	Any team member identifies a need to add, modify, or remove a requirement	Any team member
2	Log on ClickUp	Requester creates a ClickUp task with: requirement ID, change description, reason, and estimated impact on WBS or schedule	Requester
3	Impact Analysis	Affected team member(s) assess implementation impact; QA Engineer assesses verification impact	Shace Tariq / Ali Sajjad / Hamad Khan
4	Approval	Team Lead approves internal changes. Changes affecting API contracts additionally require Integration Group approval via ClickUp.	Shah Faisal (+ Integration Group if API-related)
5	Update SP-SRS & RTM	SP-SRS updated with revised requirement; RTM updated to reflect new WBS, owner, and verification details	Shah Faisal (Team Lead)
6	Communicate	Updated API contract or event schema shared with all affected module teams via ClickUp	Shah Faisal (Team Lead)

6.2 Change Log

CR-ID	Date	Req. ID	Change Description	Status
–	–	–	No changes recorded at initial baseline	–

7. Requirements Verification and Validation

7.1 Verification Responsibility

The QA Engineer (Hamad Khan) is responsible for planning and executing all verification activities. The Team Lead reviews and approves the test results before each milestone sign-off. Other team members support testing within their respective areas.

7.2 Verification Methods

Method	Description	Applied To	Executed By
Unit Testing	Individual backend functions tested in isolation using Jest or Mocha	FR-EI, FR-AG, FR-AP	Shah Faisal + Hamad Khan
Integration Testing	Module tested end-to-end with mock events from other modules; all API endpoints verified with Postman test collections	FR-EI, FR-AP	Hamad Khan
System Demo	Live demonstration of React dashboard connected to backend to Integration Group and instructor	FR-DB, FR-RP	Ali Sajjad + Hamad Khan
Performance Test	API response times and dashboard load times measured using Postman runner and browser dev tools	NFR-001, NFR-002	Hamad Khan
Inspection	Code and configuration reviewed for MERN compliance, no hardcoded secrets, and zero paid tools	NFR-003, NFR-004, NFR-005	Shace Tariq + Hamad Khan
Manual UI Test	Dashboard manually tested in Chrome and Firefox at standard viewport widths	NFR-006, FR-DB-001	Hamad Khan
Analysis	CHS algorithm reviewed manually with known inputs to verify formula correctness and output range (0-100)	FR-AG-003	Hamad Khan + Shah Faisal

7.3 Acceptance Criteria

- All High-priority functional requirements must be implemented and verified before integration handover to the Integration Group.
- At least 60% of backend aggregation and API logic covered by unit tests (Jest/Mocha) -- tracked by Hamad Khan.
- All agreed API endpoints return correct HTTP status codes and JSON response schemas as verified by Postman.
- React dashboard renders without console errors in Chrome and Firefox at 1280px+ width.
- Zero paid tools or services used in the final deployed system -- verified by Shah Faisal.
- All High and Medium NFRs verified before the final system demo.
- All requirements marked as "High Priority" must be verified and demonstrated during the final integration demo to be considered complete.

8. Requirements Management Tools and Storage

8.1 Tools

Tool	Purpose	Owner / Notes
ClickUp	Requirements tracking, change request logging, task assignment, inter-team communication	Shah Faisal (Team Lead) -- primary tool per Project Charter
GitHub	Version control for SP-SRS, SP-RMP, and SP-SDS documents and source code	Shace Tariq (Backend Dev) -- shared with Integration Group
SP-SRS Document	Primary requirements repository with full attribute table and requirement descriptions	Maintained by Ali Sajjad; submitted as separate Assignment 2 document
RTM (Section 5.2)	Maps every requirement to source, WBS, team owner, and verification method	Maintained by Hamad Khan (QA Engineer)
Postman	API requirement verification, integration testing, and performance testing	Hamad Khan (QA Engineer) -- free tier

8.2 Document Control

- The SP-RMP and SP-SRS are version-controlled on GitHub in the Analytics module repository.
- The latest version of both documents is shared with the Integration Group via ClickUp.
- Document updates follow the change management process in Section 6 and require Team Lead approval.
- All team members have read access to requirement documents; only the Team Lead may approve changes to the baseline.

9. Authorization

This SP-RMP has been prepared by the Analytics & Community Insights module team. All team members have reviewed and agreed to this requirements management plan. It is submitted as part of Assignment 2 for NUCES Section A.

TEAM LEAD

Name: Shah Faisal

Roll No.: 23I-0058

Role: Team Lead

Signature: _____

Date: 27 March 2026

REVIEWED BY (Integration Group)

Name: Hassan Qureshi

Roll No.: 23I-3029

Role: Integration Manager

Signature: _____

Date: _____

BACKEND DEVELOPER

Name: Shace Tariq

Roll No.: 23I-3069

Signature: _____

Date: _____

FRONTEND DEVELOPER

Name: Ali Sajjad

Roll No.: 23I-0110

Signature: _____

Date: _____

QA ENGINEER

Name: Hamad Khan

Roll No.: 23I-3095

Signature: _____

Date: _____